



TechBiz Academy

Week 4_Task 11

How to Web Works + Burp suite Intro

Submitted By

Ali Hamza

Registration No

TBS-2026-017

Course

Ethical Hacking Internship

Submission Date

24 July 2026

Organization

TechBizz Security

Table of Contents

Sr. No.	Contents	Page No.
1	Introduction	1
2	Objectives	2
3	Tools and Resources Used	3
4	Understanding HTTP Requests and Responses	4
4.1	HTTP Request Structure (Headers & Body)	5
4.2	HTTP Response Structure	6
5	Burp Suite Configuration	7
5.1	Setting the Browser Proxy	8
6	Intercepting, Modifying, and Forwarding HTTP Requests (DVWA)	9
7	Difference Between GET and POST Methods	11
7.1	GET Request Example	12
7.2	POST Request Example	13
8	Observations	14
9	Conclusion	15
10	References	16

1. Objective

The objective of this lab was to understand the basics of web application communication using HTTP/HTTPS, learn how HTTP requests and responses are structured, configure Burp Suite as a proxy, intercept and modify HTTP requests, and understand the differences between GET and POST methods.

2. Tools Used

- Kali Linux
 - Burp Suite Community Edition
 - DVWA (Damn Vulnerable Web Application)
 - Docker
 - Firefox Browser
 - Terminal
-

3. Understanding HTTP Requests and Responses

HTTP (HyperText Transfer Protocol) is the protocol used for communication between a web browser and a web server. Every interaction consists of a request sent by the client and a response returned by the server.

HTTP Request Structure

An HTTP request contains:

- Request Method (GET/POST)
- URL
- HTTP Version
- Headers
- Body (only in POST requests)

Example HTTP Request

```
POST /login.php HTTP/1.1
Host: 127.0.0.1:8081
Content-Type: application/x-www-form-urlencoded

username=admin&password=password
```

HTTP Response Structure

An HTTP response contains:

- Status Line
- Response Headers

- Response Body

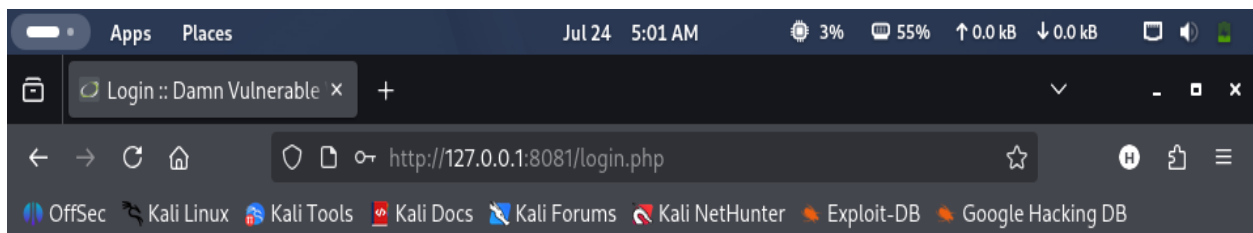
Example HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html
```

```
<html>
Login Successful
</html>
```

Screenshot 1

DVWA Login Page



Username

Password

4. Configuring Burp Suite

Burp Suite Community Edition was launched from Kali Linux.

The browser proxy was configured to route all HTTP traffic through Burp Suite so that requests could be intercepted and modified before reaching the server.

Steps performed:

1. Opened Burp Suite.
2. Navigated to **Proxy** → **Intercept**.
3. Enabled **Intercept ON**.
4. Opened the browser through Burp Suite.
5. Configured traffic to pass through the proxy.

Screenshot 2

Burp Suite Proxy Configuration

The screenshot displays the Burp Suite Community Edition v2026.2.3 interface. The 'Proxy' tab is selected, and the 'Intercept' sub-tab is active, showing 'Intercept on' as the selected action. A table of intercepted HTTP requests is visible, with the following data:

Time	Type	Direction	Method	URL	Status code	Length
02:58:5...	HTTP	→ Request	GET	https://www.google.com/warmup.html		
02:59:0...	HTTP	→ Request	GET	https://httpbin.org/get		
02:59:1...	HTTP	→ Request	GET	https://www.google.com/warmup.html		
03:02:0...	HTTP	→ Request	GET	https://httpbin.org/get		

The 'Request' pane shows the details of the selected request (GET /warmup.html). The 'Inspector' pane shows the request attributes, including request query parameters, request body parameters, request cookies, and request headers.

Event log (1) All issues

Memory: 77.4MB of 1.09GB

5. Installing and Running DVWA

Docker was installed and verified.

Docker service was started successfully.

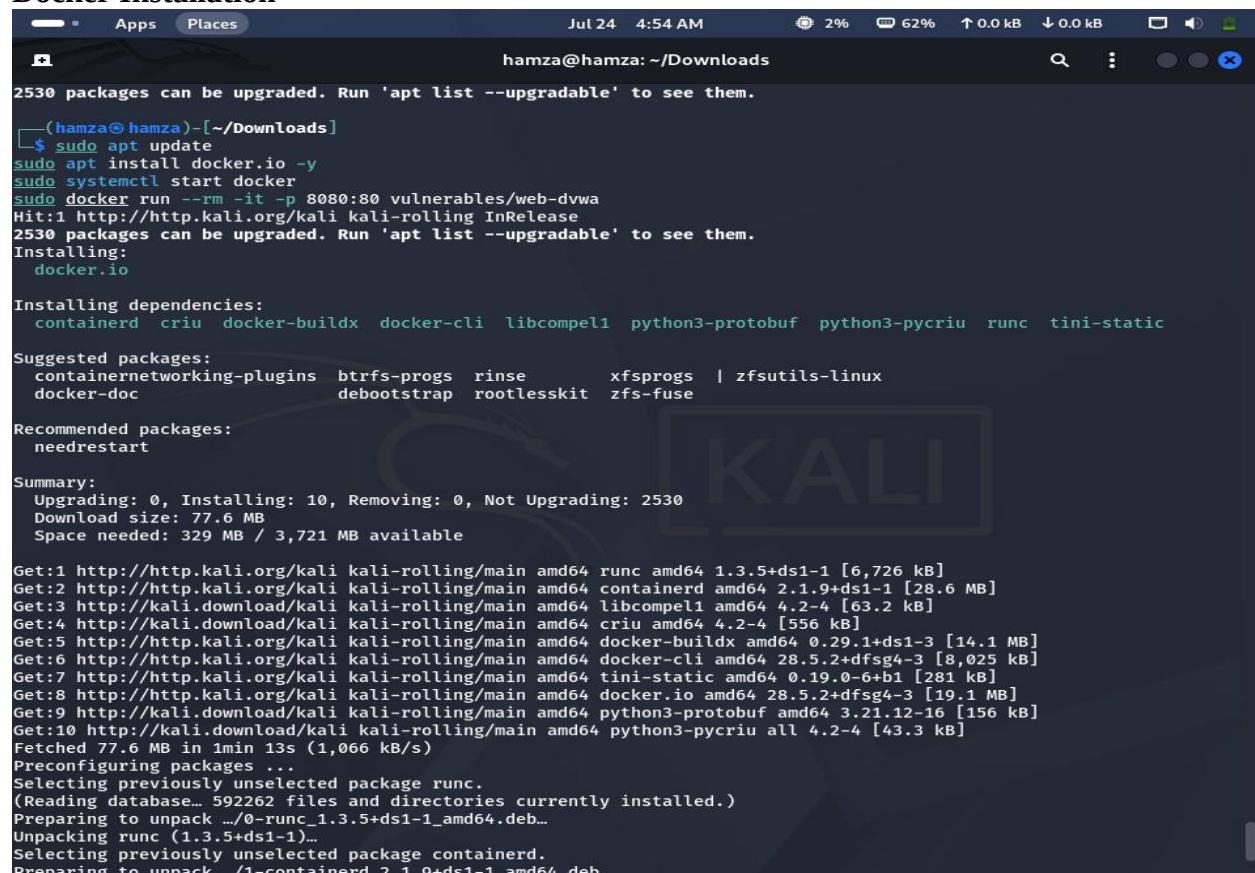
The DVWA container was executed using:

```
sudo docker run --rm -it -p 8081:80 vulnerables/web-dvwa
```

Initially, port 8080 was already occupied, therefore port **8081** was used successfully.

Screenshot 3

Docker Installation



```
hamza@hamza: ~/Downloads
2530 packages can be upgraded. Run 'apt list --upgradable' to see them.

(hamza@hamza)~[/Downloads]
$ sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo docker run --rm -it -p 8080:80 vulnerables/web-dvwa
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2530 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  docker.io

Installing dependencies:
  containerd criu docker-buildx docker-cli libcompel1 python3-protobuf python3-pycrui runc tini-static

Suggested packages:
  containernetworking-plugins btrfs-progs rinse xfsprogs | zfsutils-linux
  docker-doc debotstrap rootlesskit zfs-fuse

Recommended packages:
  needrestart

Summary:
  Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 2530
  Download size: 77.6 MB
  Space needed: 329 MB / 3,721 MB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 runc amd64 1.3.5+ds1-1 [6,726 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 containerd amd64 2.1.9+ds1-1 [28.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 libcompel1 amd64 4.2-4 [63.2 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 criu amd64 4.2-4 [556 kB]
Get:5 http://http.kali.org/kali kali-rolling/main amd64 docker-buildx amd64 0.29.1+ds1-3 [14.1 MB]
Get:6 http://http.kali.org/kali kali-rolling/main amd64 docker-cli amd64 28.5.2+dfsg4-3 [8,025 kB]
Get:7 http://http.kali.org/kali kali-rolling/main amd64 tini-static amd64 0.19.0-6+b1 [281 kB]
Get:8 http://http.kali.org/kali kali-rolling/main amd64 docker.io amd64 28.5.2+dfsg4-3 [19.1 MB]
Get:9 http://kali.download/kali kali-rolling/main amd64 python3-protobuf amd64 3.21.12-16 [156 kB]
Get:10 http://kali.download/kali kali-rolling/main amd64 python3-pycrui all 4.2-4 [43.3 kB]
Fetched 77.6 MB in 1min 13s (1,066 kB/s)
Preconfiguring packages ...
Selecting previously unselected package runc.
(Reading database... 592262 files and directories currently installed.)
Preparing to unpack .../0-runc-1.3.5+ds1-1_amd64.deb...
Unpacking runc (1.3.5+ds1-1)...
Selecting previously unselected package containerd.
Preparing to unpack .../1-containerd-2.1.9+ds1-1_amd64.deb...
```

Screenshot 4

Docker Service Status

```
hamza@hamza: ~/Downloads
└─$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(hamza@hamza)-[~/Downloads]
└─$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-07-24 03:22:44 EDT; 1h 32min ago
     Invocation: efb441fb20b64c7184d224779f007d87
   TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
     Main PID: 87541 (dockerd)
        Tasks: 11
       Memory: 601.4M (peak: 832M)
          CPU: 39.139s
       CGroup: /system.slice/docker.service
               └─87541 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jul 24 03:22:44 hamza dockerd[87541]: time="2026-07-24T03:22:44.352805495-04:00" level=info msg="Docker daemon" comm>
Jul 24 03:22:44 hamza dockerd[87541]: time="2026-07-24T03:22:44.352970774-04:00" level=info msg="Initializing buildk>
Jul 24 03:22:44 hamza dockerd[87541]: time="2026-07-24T03:22:44.380832750-04:00" level=info msg="Completed buildkit >
Jul 24 03:22:44 hamza dockerd[87541]: time="2026-07-24T03:22:44.403696929-04:00" level=info msg="Daemon has complete>
Jul 24 03:22:44 hamza dockerd[87541]: time="2026-07-24T03:22:44.403910916-04:00" level=info msg="API listen on /run/>
Jul 24 03:22:44 hamza systemd[1]: Started docker.service - Docker Application Container Engine.
Jul 24 03:27:37 hamza dockerd[87541]: time="2026-07-24T03:27:37.607463930-04:00" level=warning msg="Failed to allocat>
Jul 24 03:27:37 hamza dockerd[87541]: time="2026-07-24T03:27:37.679326064-04:00" level=info msg="ignoring event" con>
Jul 24 03:27:37 hamza dockerd[87541]: time="2026-07-24T03:27:37.744459822-04:00" level=error msg="Handler for POST />
Jul 24 03:27:38 hamza dockerd[87541]: time="2026-07-24T03:27:38.595560682-04:00" level=warning msg="MAC address chan>

(hamza@hamza)-[~/Downloads]
└─$ sudo docker run --rm -it -p 8080:80 vulnerables/web-dvwa
docker: Error response from daemon: failed to set up container networking: driver failed programming external connect
ivity on endpoint frosty_hofstadter (241a4bded5c996464216b7e95cae2644ec8a97d8476f7a9e6f0c67ecc07454c8): failed to bin
d host port for 0.0.0.0:8080:172.17.0.2:80/tcp: address already in use

Run 'docker run --help' for more information

(hamza@hamza)-[~/Downloads]
└─$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(hamza@hamza)-[~/Downloads]
└─$
```

6. Intercepting and Modifying Requests

After logging into DVWA:

- Security Level was set to **Low**.
- Burp Suite intercepted the HTTP request.
- The intercepted request was examined.
- The request parameter was modified before forwarding it to the web server.

Original Request

```
POST /security.php HTTP/1.1  
  
security=low
```

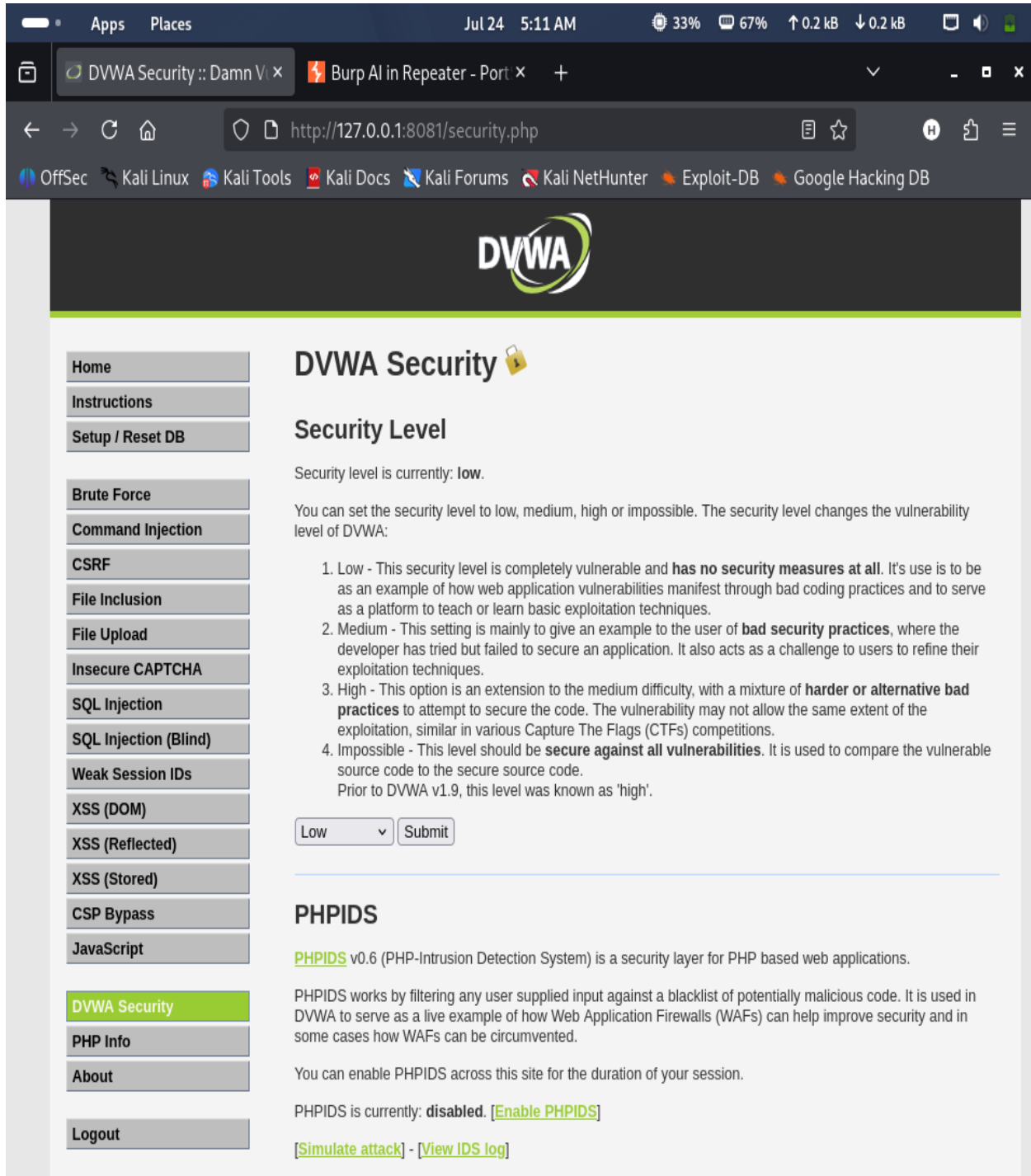
Modified Request

```
POST /security.php HTTP/1.1  
  
security=high
```

The modified request was forwarded successfully using Burp Suite.

Screenshot 5

DVWA Security Page



The screenshot shows a web browser window with the following details:

- Browser Tabs:** DVWA Security :: Damn V..., Burp AI in Repeater - Port x
- Address Bar:** http://127.0.0.1:8081/security.php
- Browser Extensions:** OffSec, Kali Linux, Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, Exploit-DB, Google Hacking DB
- Page Header:** DVWA logo
- Sidebar Navigation:**
 - Home
 - Instructions
 - Setup / Reset DB
 - Brute Force
 - Command Injection
 - CSRF
 - File Inclusion
 - File Upload
 - Insecure CAPTCHA
 - SQL Injection
 - SQL Injection (Blind)
 - Weak Session IDs
 - XSS (DOM)
 - XSS (Reflected)
 - XSS (Stored)
 - CSP Bypass
 - JavaScript
 - DVWA Security**
 - PHP Info
 - About
 - Logout
- Main Content Area:**

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

 1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
 2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
 3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
 4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low [Submit]

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Screenshot 6

Burp Suite Intercepting Request

The screenshot displays the Burp Suite Community Edition v2026.2.3 interface. The top menu bar includes options like Apps, Places, and a status bar showing system information (Jul 24, 5:13 AM, 5% CPU, 67% RAM, 0.0 kB up/down). The main toolbar features tabs for Dashboard, Target, Proxy (selected), Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, and Logger. Below this, there are sub-tabs for Interceptor (selected), HTTP history, WebSockets history, Match and replace, and Proxy settings.

The Interceptor panel shows a list of intercepted requests. The first request is highlighted:

Time	Type	Direction	Method	URL	Status code	Le
05:02:...	HT...	→	Request	GET https://www.google.com/warmup.html		
05:03:...	HT...	→	Request	GET http://127.0.0.1:8081/		
05:05:...	HT...	→	Request	GET http://127.0.0.1:8081/		
05:07:...	HT...	→	Request	GET http://127.0.0.1:8081/		
05:07:...	HT...	→	Request	GET http://127.0.0.1:8081/		

The bottom section of the interface is split into two panels. The left panel, titled 'Request', shows the raw HTTP request details:

```
1 GET /warmup.html HTTP/1.1
2 Host: www.google.com
3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
4 Sec-Ch-Ua-Mobile: ?0
5 Sec-Ch-Ua-Platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
```

The right panel, titled 'Inspector', shows the request attributes and headers:

Inspector	Count	Expand
Request attributes	2	▼
Request query parameters	0	▼
Request body parameters	0	▼
Request cookies	0	▼
Request headers	17	▼

The bottom status bar shows 'Event log (1)' and 'All issues', along with memory usage: 'Memory: 191.9MB of 1.09GB' and a 'Disabled' toggle.

7. Difference Between GET and POST

GET

Retrieves data from the server.

Parameters are included in the URL.

Less secure for sensitive information.

Used for reading data.

POST

Sends data to the server.

Parameters are included in the request body.

More secure because data is not visible in the URL.

Used for submitting forms and login credentials.

GET Example

```
GET /search?q=dvwa HTTP/1.1
Host: example.com
```

URL:

```
https://example.com/search?q=dvwa
```

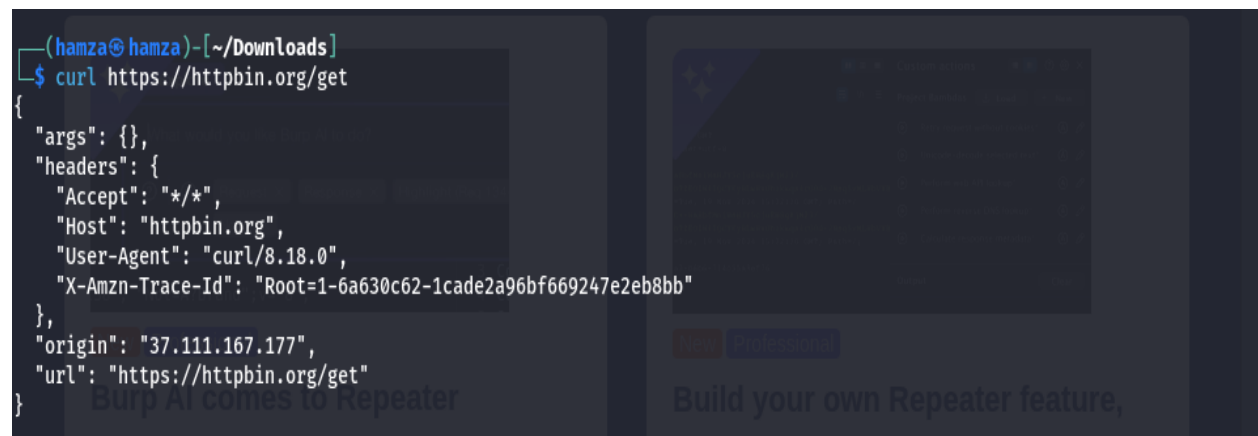
POST Example

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

username=admin&password=password
```

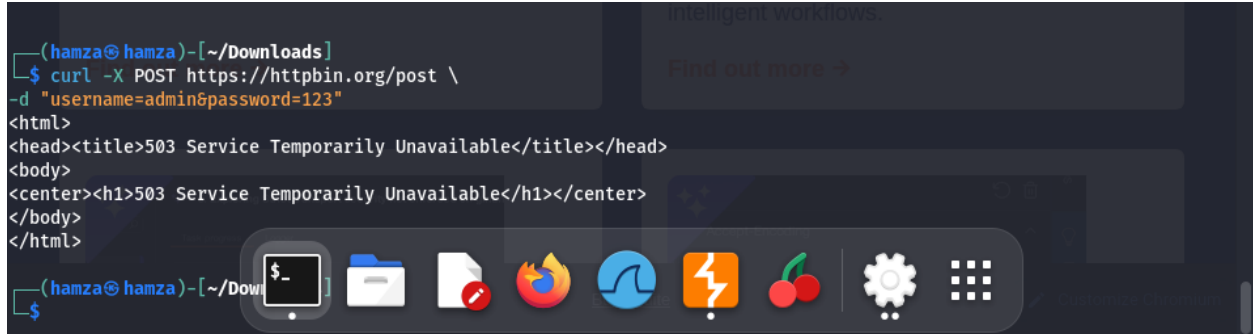
Screenshot 7

HTTP GET Request Example



Screenshot 8

HTTP POST Request Example



```
(hamza@hamza) - [~/Downloads]
$ curl -X POST https://httpbin.org/post \
-d "username=admin&password=123"
<html>
<head><title>503 Service Temporarily Unavailable</title></head>
<body>
<center><h1>503 Service Temporarily Unavailable</h1></center>
</body>
</html>
```

8. Observations

- Successfully configured Burp Suite as a proxy.
- Learned the structure of HTTP requests and responses.
- Successfully intercepted HTTP traffic.
- Modified request parameters before forwarding them to the server.
- Successfully deployed DVWA using Docker.
- Understood the practical difference between GET and POST methods.

Resources Used

1. TryHackMe – Burp Suite: The Basics

This free TryHackMe room introduces the fundamentals of Burp Suite. It explains how to configure Burp Suite as a proxy, intercept HTTP/HTTPS requests, inspect and modify requests and responses, and use the basic tools such as Proxy, HTTP History, Repeater, and Intruder. The room provides hands-on practice in understanding web application traffic.

2. TryHackMe – Walking An Application

This room teaches the fundamentals of exploring a web application from a penetration tester's perspective. It covers website structure, URLs, parameters, directories, forms, cookies, and client-server communication. It helps learners understand how web applications work before performing security testing.

Video Resources

1. Burp Suite Tutorial for Beginners

This tutorial demonstrates how to install and configure Burp Suite, set up the browser proxy, intercept HTTP requests, modify request parameters, and forward requests to the web server. It provides practical examples of using Burp Suite for web application security testing.

2. How HTTP Works – Request & Response Explained

This video explains the working of the HTTP protocol, including the structure of HTTP requests and responses, request methods (GET and POST), headers, body, status codes, cookies, and sessions. It provides a clear understanding of how browsers and web servers communicate during web requests.